

From Digital Logic to Digital Circuits

Previous Week: Digital Logic Foundations

We developed the foundations for representing and manipulating information using digital logic.

- **Data Representation**

- Number systems
- Binary arithmetic and complements
- BCD and Gray code

- **Logic Gates**

- AND, OR, NOT, NAND, NOR, XOR
- Functionally complete gates

- **Boolean Algebra**

- Boolean laws and simplification
- Boolean functions

- **Function Representation**

- Minterms and maxterms
- SOP and POS
- Karnaugh maps

This Week

We now use **Boolean functions** and **logic gates** to build actual **digital circuits**.

Digital Logic



Digital Circuits

Types of Digital Circuits

Classification

Digital Circuits $\longrightarrow$ **Combinational** **Sequential**

Combinational Circuits

Output depends only on the **present inputs**.

$$Y = f(X)$$

Key idea: No memory

Examples:

- Adders / Subtractors
- Multiplexers
- Encoders / Decoders

Sequential Circuits

Output depends on the **present inputs and stored state**.

$$Y = f(X, Q)$$

Key idea: Has memory / state

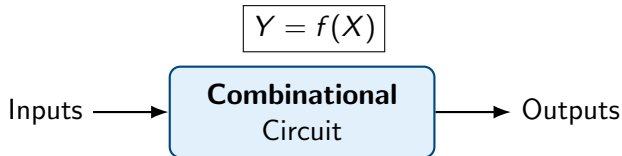
Examples:

- Latches / Flip-Flops
- Registers
- Counters

Combinational Circuits

Definition

A **combinational circuit** is a digital circuit whose output depends only on the **present inputs**.



Common Examples

- Adders and subtractors
- Multiplexers
- Decoders / Demultiplexers
- Encoders

Full Adder

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum

$$S_i = A_i \oplus B_i \oplus C_i$$

Carry

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$

Carry Intuition

Carry is 1 when at least **two of the three inputs are 1**.

Understanding Carry Generation and Propagation

A_i	B_i	$G_i = A_i B_i$	$P_i = A_i \oplus B_i$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Generate

If $A_i = B_i = 1$:

The stage generates a carry by itself.

$$G_i = 1$$

Propagate

$$A_i + B_i = 1$$

An incoming carry passes forward.

$$P_i = 1$$

$$C_{i+1} = G_i + P_i C_i$$

Why is Carry Propagation Important?

For every bit position,

$$C_{i+1} = G_i + P_i C_i$$

For a multi-bit adder:

$$C_1 = G_0 + P_0 C_0,$$

$$C_2 = G_1 + P_1 C_1,$$

$$C_3 = G_2 + P_2 C_2,$$

$$C_4 = G_3 + P_3 C_3.$$

What Happens?

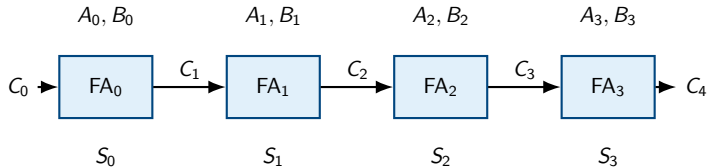
To determine C_2 , we first need C_1 .

To determine C_3 , we first need C_2 .

To determine C_4 , we first need C_3 .

From Full Adder to Ripple Carry Adder

An n -bit adder can be built by cascading Full Adders.



Ripple of Carry

FA₀ produces C₁ → FA₁ produces C₂ → FA₂ produces C₃ → FA₃ produces C₄.

Main Limitation

Each stage may have to wait for the carry from the previous stage.

More bits $\Rightarrow$ longer carry delay

Can the Same Circuit Perform Addition and Subtraction?

We already have an n -bit adder for

$$A + B$$

But in 2's-complement arithmetic,

$$A - B = A + (-B)$$

and

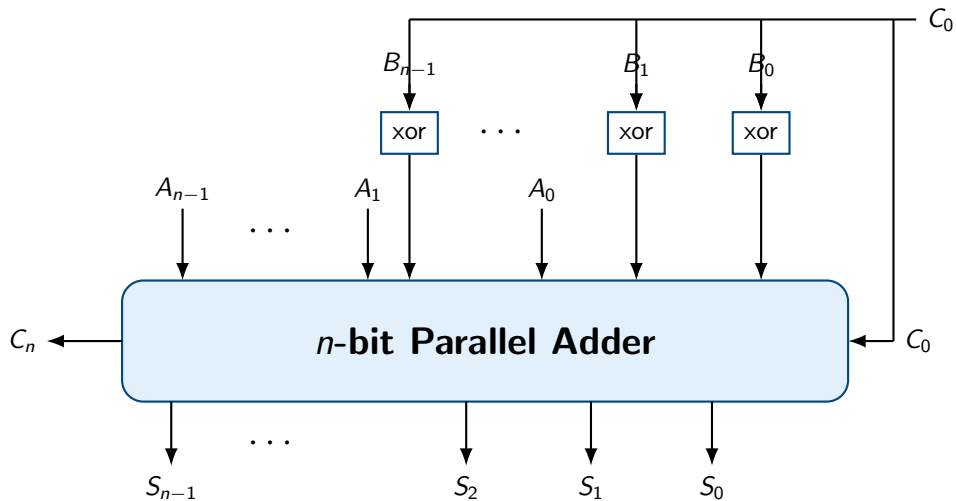
$$-B = \overline{B} + 1$$

Therefore,

$$A - B = A + \overline{B} + 1$$

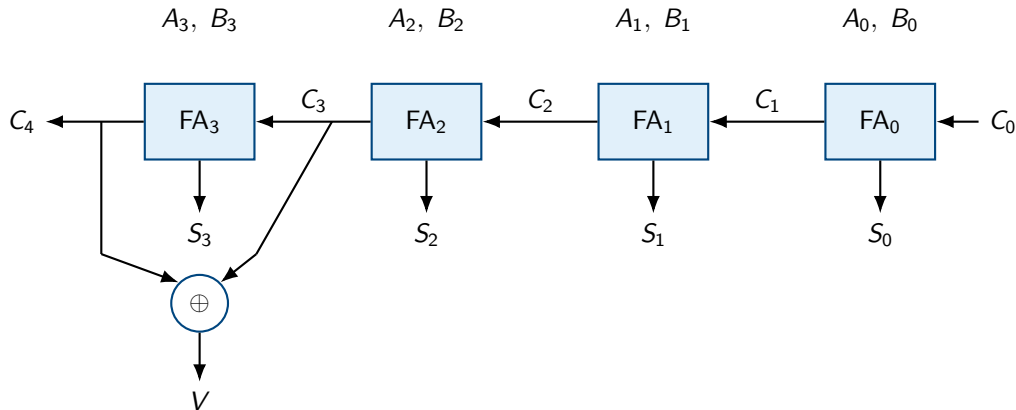
Operation	What the Adder Must Compute
Addition	$A + B + 0$
Subtraction	$A + \overline{B} + 1$

A Parallel Adder / Subtractor



4-bit Adder: Detecting Signed Overflow

For 4-bit 2's-complement addition, connect four Full Adders:



$$V = C_3 \oplus C_4$$

Example: 4-bit 2's-Complement Overflow

Consider:

$$7 + 3$$

4-bit representation:

But:

$$7 = 0111, \quad 3 = 0011$$

$$7 + 3 = 10$$

$$\begin{array}{r} 0111 \\ +0011 \\ \hline 1010 \end{array}$$

4-bit 2's-complement range:

$$\boxed{-8 \leq x \leq 7}$$

For the MSB Full Adder:

Therefore, 10 cannot be represented.

$$C_3 = 1, \quad C_4 = 0$$

Hence,

$$\boxed{V = C_3 \oplus C_4 = 1 \oplus 0 = 1}$$

Why Do We Need a Carry Look-Ahead Adder?

In a Ripple Carry Adder:

$$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \dots \rightarrow C_n$$

Each stage may have to wait for the carry from the previous stage.

Ripple Carry Adder	Carry Look-Ahead Adder
Carry generated sequentially Delay grows with n Simple hardware	Carry signals generated in parallel Much smaller carry delay More hardware complexity

Main idea of CLA:

Compute the carries directly instead of waiting for them to ripple.

The Key Idea Behind Carry Look-Ahead

For an RCA,

$$C_2 = G_1 + P_1 C_1$$

so it appears that we must first wait for C_1 .

But

$$C_1 = G_0 + P_0 C_0$$

Substituting:

$$\begin{aligned} C_2 &= G_1 + P_1(G_0 + P_0 C_0) \\ &= G_1 + P_1 G_0 + P_1 P_0 C_0 \end{aligned}$$

Therefore,

$$\boxed{C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0}$$

C_2 can now be calculated directly from G , P , and C_0 .

Carry Look-Ahead: 4-bit Carry Equations

Starting with

$$C_{i+1} = G_i + P_i C_i$$

we obtain:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

Carry Look-Ahead: Final Carry

For the most significant carry:

$$\begin{aligned} C_4 = & G_3 + P_3 G_2 \\ & + P_3 P_2 G_1 \\ & + P_3 P_2 P_1 G_0 \\ & + P_3 P_2 P_1 P_0 C_0 \end{aligned}$$

Notice that C_4 depends directly on

$$G_3, G_2, G_1, G_0, \quad P_3, P_2, P_1, P_0, \quad C_0$$

and not on waiting sequentially for

$$C_1, C_2, C_3.$$

This is the “look-ahead” in Carry Look-Ahead Adder.

Generating the Sum Bits

Recall:

$$P_i = A_i \oplus B_i$$

The Full Adder sum is

$$S_i = A_i \oplus B_i \oplus C_i$$

Therefore,

$$S_i = P_i \oplus C_i$$

For a 4-bit adder:

$$S_0 = P_0 \oplus C_0$$

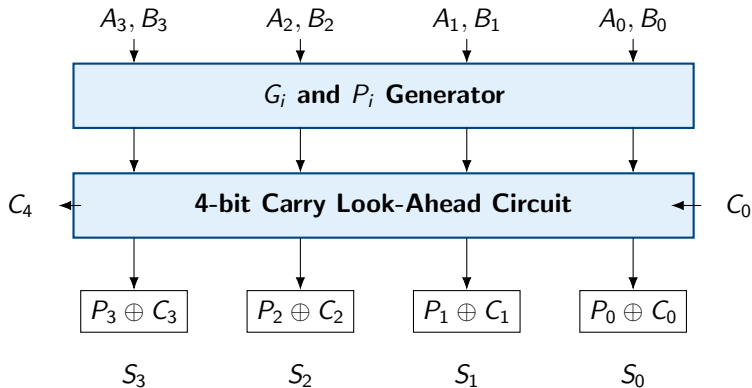
$$S_1 = P_1 \oplus C_1$$

$$S_2 = P_2 \oplus C_2$$

$$S_3 = P_3 \oplus C_3$$

Once the carries are available, all sum bits can be generated using XOR gates.

4-bit Carry Look-Ahead Adder



Ripple Carry Adder vs Carry Look-Ahead Adder

	RCA	CLA
Carry calculation	Sequential	Parallel
Hardware	Simple	More complex
Propagation delay	Grows with n	Much smaller
Main advantage	Low hardware cost	High speed

RCA:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

CLA:

C_1, C_2, C_3, C_4 computed directly from G_i, P_i, C_0

Carry Select Adder: Basic Idea

Problem: In an RCA, a block cannot complete its addition until its input carry is known.

Why wait for the carry?

Suppose the incoming carry can only be

$$C_{in} = 0 \quad \text{or} \quad C_{in} = 1$$

Compute **both possibilities in advance**:

$$C_{in} = 0 \Rightarrow (S^{(0)}, C_{out}^{(0)})$$

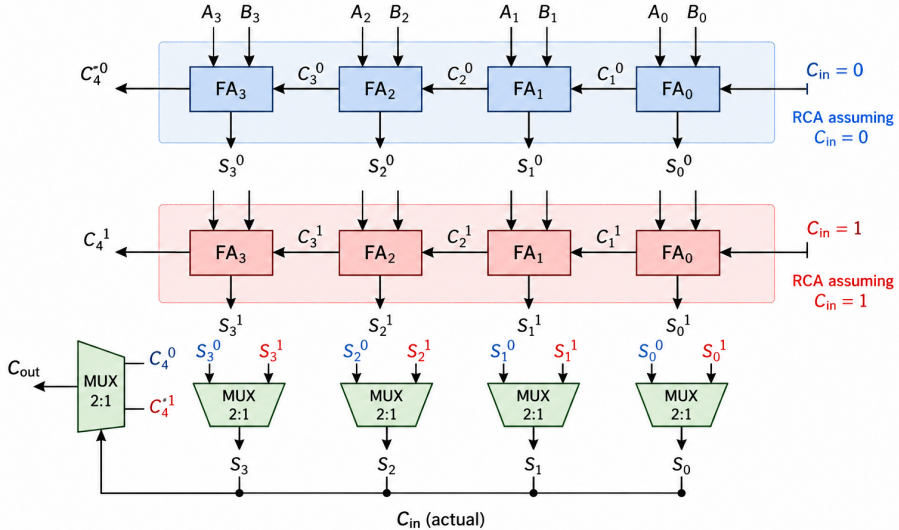
$$C_{in} = 1 \Rightarrow (S^{(1)}, C_{out}^{(1)})$$

When the actual carry arrives, use a multiplexer:

$C_{in} \longrightarrow \text{MUX} \longrightarrow \text{Correct Result}$

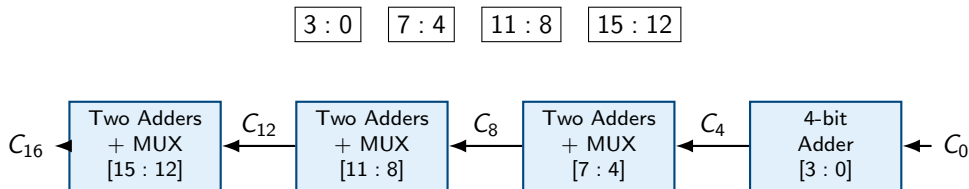
Instead of waiting for the carry, calculate both cases.

Basic Building Block of a Carry Select Adder



16-bit Carry Select Adder

Divide the 16-bit adder into four 4-bit blocks:



The least significant block needs only **one adder**, because C_0 is already known.

$$T_{approx} = 4 T_{FA} + 3 T_{MUX}$$

The carry selects a precomputed result instead of rippling through every Full Adder.

Can Carry Select Blocks Have Different Sizes?

Yes. Carry Select blocks need not have equal sizes.

Uniform blocks:

$$4 + 4 + 4 + 4 = 16$$

Example of variable blocks:

$$2 + 2 + 3 + 4 + 5 = 16$$

Why increase the block size?

- Earlier blocks start computation earlier.
- Later blocks have more time while the carry is being selected.
- Therefore, later blocks can contain more bits.

Block size can be chosen to balance adder and MUX delays.

Beyond Adders: Data Selection and Distribution

So far, we have designed circuits that perform **arithmetic operations**:

Half Adder → Full Adder → RCA → CLA → Carry Select Adder

But a computer must also decide:

Which data should be used?

and

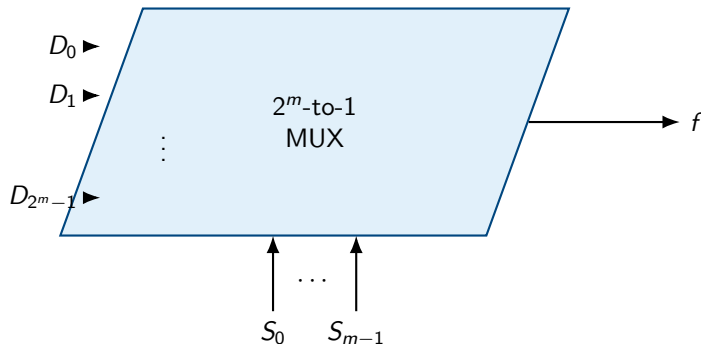
Where should the data be sent?

MUX: Many Inputs → One Output

DEMUX: One Input → Many Outputs

Multiplexer (MUX): Data Selector

A **Multiplexer** selects **one of many inputs** and connects it to a single output.



m select lines can select one of 2^m inputs

2-to-1 Multiplexer

A 2-to-1 MUX has

2 data inputs,

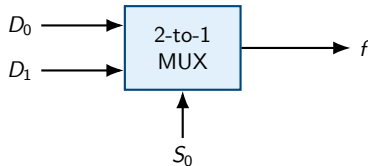
1 select input,

1 output.

S_0	f
0	D_0
1	D_1

Therefore,

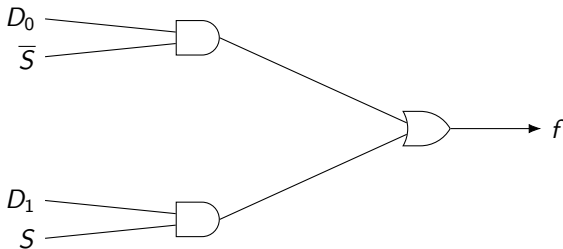
$$f = \overline{S_0}D_0 + S_0D_1$$



2-to-1 MUX using Logic Gates

From the MUX equation:

$$f = \overline{S}D_0 + SD_1$$



$S = 0 \Rightarrow f = D_0$	$S = 1 \Rightarrow f = D_1$
-----------------------------	-----------------------------

4-to-1 Multiplexer

Four data inputs require

$$\log_2 4 = 2$$

select lines.

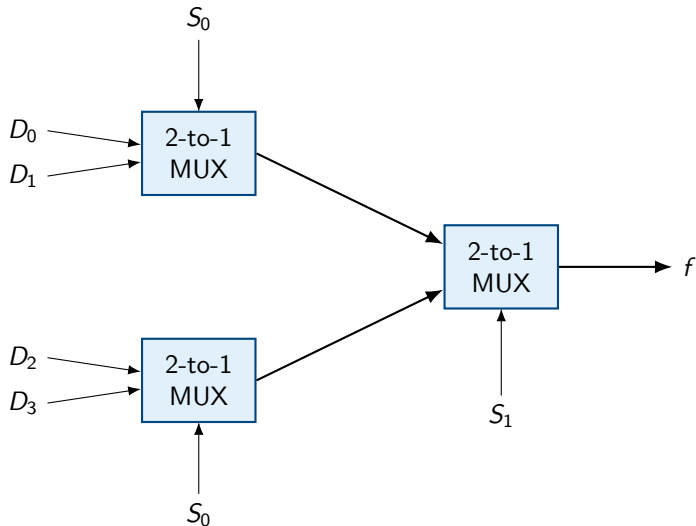
S_1	S_0	Output f
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Hence,

$$f = \overline{S_1}\overline{S_0}D_0 + \overline{S_1}S_0D_1 + S_1\overline{S_0}D_2 + S_1S_0D_3$$

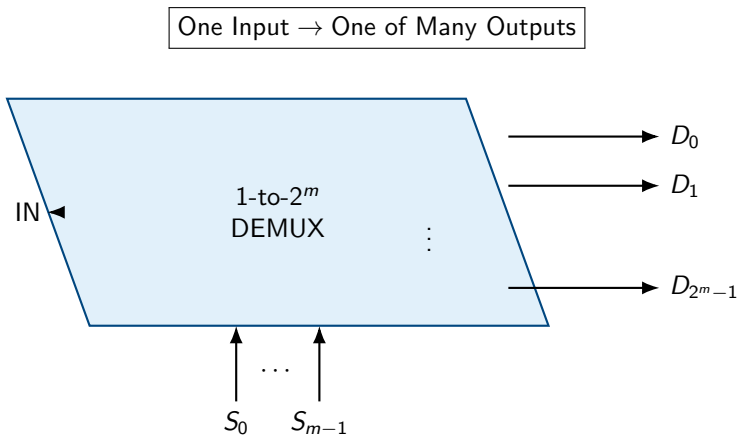
Select lines behave exactly like an address: they specify which input is connected to the output.

Building a 4-to-1 MUX using 2-to-1 MUXes



Demultiplexer (DEMUX): Data Distributor

A **Demultiplexer** performs the reverse operation of a MUX.



The select lines determine **where the input is sent**.

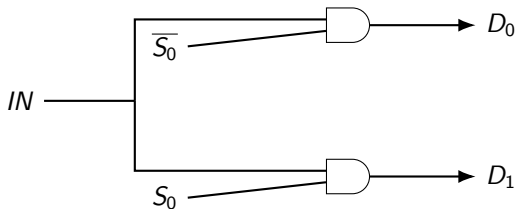
1-to-2 Demultiplexer

A 1-to-2 DEMUX has one input, one select line, and two outputs.

S_0	D_0	D_1
0	IN	0
1	0	IN

Therefore,

$$D_0 = \overline{S_0} \cdot IN, \quad D_1 = S_0 \cdot IN$$



1-to-4 Demultiplexer

Two select lines determine which one of four outputs receives the input.

S_1	S_0	D_0	D_1	D_2	D_3
0	0	IN	0	0	0
0	1	0	IN	0	0
1	0	0	0	IN	0
1	1	0	0	0	IN

$$D_0 = \overline{S_1} \overline{S_0} IN$$

$$D_1 = \overline{S_1} S_0 IN$$

$$D_2 = S_1 \overline{S_0} IN$$

$$D_3 = S_1 S_0 IN$$

Notice

If $IN = 1$, these equations become exactly the minterms of $S_1 S_0$.

MUX vs DEMUX

	MUX	DEMUX
Data inputs	Many	One
Data outputs	One	Many
Select lines	m	m
Purpose	Select data	Distribute data
Direction	$2^m \rightarrow 1$	$1 \rightarrow 2^m$

$$D_0, D_1, \dots, D_{2^m-1} \xrightarrow{MUX} f$$

$$IN \xrightarrow{DEMUX} D_0, D_1, \dots, D_{2^m-1}$$

Next question: What happens if we fix $IN = 1$ in a DEMUX?

Decoder

Shannon Decomposition and Multiplexers

Any Boolean function can be decomposed with respect to one variable, say x :

$$f(x, \mathbf{y}) = \bar{x} f(0, \mathbf{y}) + x f(1, \mathbf{y})$$

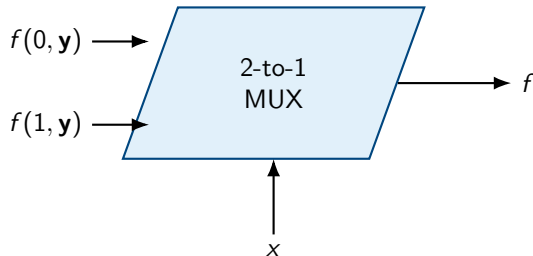
where $\mathbf{y}$ represents the remaining variables.

Interpretation

$$x = 0 \Rightarrow f = f(0, \mathbf{y})$$

$$x = 1 \Rightarrow f = f(1, \mathbf{y})$$

So x decides which sub-function appears at the output.



Key Idea

One application of Shannon decomposition is equivalent to one **2-to-1 MUX**.

Sequential Circuits

So far, we studied **combinational circuits**:

Output depends only on present inputs

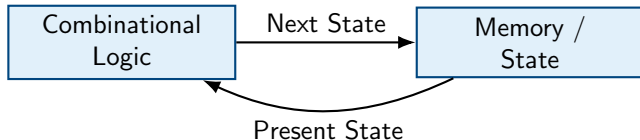
In a **sequential circuit**, the output depends on:

Present Inputs + Previous State

Therefore,

$$Y(t) = f(X(t), Q(t))$$

where $Q(t)$ represents the stored state.



Basic Idea Behind Storing a Bit

Connect two NOT gates in a feedback loop:



Two stable possibilities exist:

$$Q = 0$$

or

$$Q = 1$$

Because of feedback, the value is preserved.

But there is a problem

How do we intentionally store a new 0 or 1?

From Feedback to a Latch

The feedback loop can **remember** a bit.
But we also need a way to:

Set the stored value to 1

Reset the stored value to 0

Therefore,

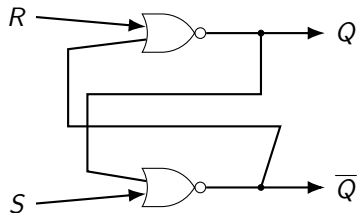
Feedback + Control Inputs $\longrightarrow$ Latch

A latch has two stable states:

0 and 1

and can therefore store **one bit**.

S-R Latch using NOR Gates



$S = \text{Set}, \quad R = \text{Reset}$

S	R	Q_{next}
0	0	Hold
0	1	0
1	0	1
1	1	Invalid

Understanding the S-R Latch

Inputs	Operation	Result
$S = 1, R = 0$	Set	$Q = 1$
$S = 0, R = 1$	Reset	$Q = 0$
$S = 0, R = 0$	Hold	$Q_{\text{next}} = Q$
$S = 1, R = 1$	Invalid	$Q = \overline{Q} = 0$

Why does Hold work?

When

$$S = R = 0$$

neither input forces a new state.

Feedback preserves the previously stored value

Why is $S = R = 1$ Invalid?

For the NOR S-R latch:

$$S = R = 1$$

forces both NOR outputs to zero:

$$Q = 0, \quad \overline{Q} = 0$$

But Q and $\overline{Q}$ should be complementary.

Now change the inputs to:

$$S = R = 0$$

Both gates try to change state.

Race Condition

The final stored state may depend on which gate responds first.

Final state may depend on gate propagation delays

S-R Latch: Excitation Table

$(Q_n, Q_{n+1}) \longrightarrow \text{Required inputs } (S, R)$

Q_n	Q_{n+1}	S	R	Operation
0	0	0	X	Hold / Reset
0	1	1	0	Set
1	0	0	1	Reset
1	1	X	0	Hold / Set

When $Q_n = 0$:

$0 \rightarrow 0 : S = 0, R = X$

$0 \rightarrow 1 : S = 1, R = 0$

When $Q_n = 1$:

$1 \rightarrow 0 : S = 0, R = 1$

$1 \rightarrow 1 : S = X, R = 0$

$X = \text{don't care}$

S-R Latch: Characteristic Equation using K-map

	SR			
	00	01	11	10
$Q_t = 0$	0	0	X	1
$Q_t = 1$	1	0	X	1

wrap-around group: $\overline{R}Q_t$

$$Q_{t+1} = S + \overline{R}Q_t$$

Valid for S and R not both equal to 1.

Gated S-R Latch

Add an **Enable** input E :

$E = 0 \Rightarrow \text{Hold}$

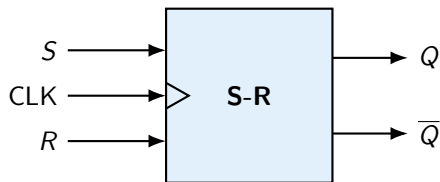
$E = 1 \Rightarrow \text{Respond to } S, R$

E	S	R	Q_{next}
0	X	X	Hold
1	0	0	Hold
1	0	1	0
1	1	0	1
1	1	1	Invalid

E decides when the latch can change.

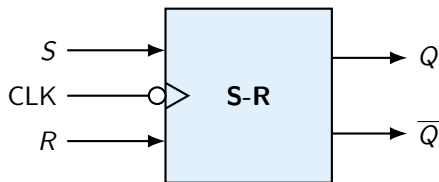
S-R Flip-Flop: Positive and Negative Edge Triggering

Positive-Edge Triggered



↑ CLK

Negative-Edge Triggered



↓ CLK

Clock Condition	S	R	Q_{next}
No active edge	X	X	Hold
Active edge	0	0	Hold
Active edge	0	1	0
Active edge	1	0	1
Active edge	1	1	Invalid

From S-R Latch to D Latch

The S-R latch has two data inputs:

$S, \quad R$

and one unwanted combination:

$S = R = 1$ Invalid

What if we use only **one data input** D ?

Choose

$S = D$

$R = \overline{D}$

Then:

$$D = 0 \Rightarrow (S, R) = (0, 1)$$

$$D = 1 \Rightarrow (S, R) = (1, 0)$$

The invalid combination can never occur.

Gated D Latch

A D latch has one data input D and an enable E .

E	D	Q_{next}
0	X	Hold
1	0	0
1	1	1

When enabled:

$$E = 1 \Rightarrow Q_{\text{next}} = D$$

When disabled:

$$E = 0 \Rightarrow Q_{\text{next}} = Q$$

The D latch stores the value present at D while it is enabled.

J-K Latch

The S-R latch has one invalid condition:

$$S = R = 1$$

The J-K latch removes this problem.

J	K	Q_{next}
0	0	Q
0	1	0
1	0	1
1	1	$\overline{Q}$

Input	Operation
$J = 0, K = 0$	Hold
$J = 0, K = 1$	Reset
$J = 1, K = 0$	Set
$J = 1, K = 1$	Toggle

How Does J-K Improve the S-R Latch?

For an S-R latch:

$$S = R = 1$$

is invalid.

In a J-K latch, feedback from the outputs is used so that the equivalent S and R inputs cannot force the invalid state.

$$S = J\overline{Q} \quad R = KQ$$

Therefore, when

$$J = K = 1,$$

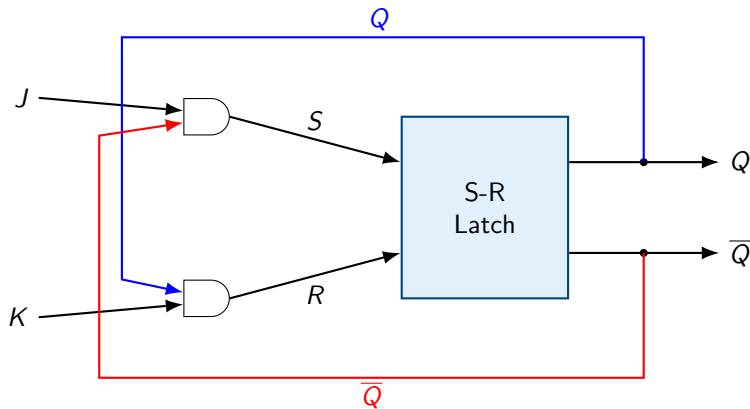
we get

$$S = \overline{Q}, \quad R = Q$$

so the next state becomes

$$Q_{\text{next}} = \overline{Q}$$

J-K Latch: Feedback Structure



$$S = J\overline{Q}, \quad R = KQ$$

J-K Latch: Excitation Table

$(Q_n, Q_{n+1}) \longrightarrow \text{Required inputs } (J, K)$

Q_n	Q_{n+1}	J	K	Operation
0	0	0	X	Hold
0	1	1	X	Set
1	0	X	1	Reset
1	1	X	0	Hold

When $Q_n = 0$:

$0 \rightarrow 0 : J = 0$

$0 \rightarrow 1 : J = 1$

When $Q_n = 1$:

$1 \rightarrow 0 : K = 1$

$1 \rightarrow 1 : K = 0$

X = don't care

J-K: Characteristic Equation using K-map

	JK			
	00	01	11	10
$Q_t = 0$	0	0	1	1
$Q_t = 1$	1	0	0	1

wrap-around group: $\overline{K}Q_t$

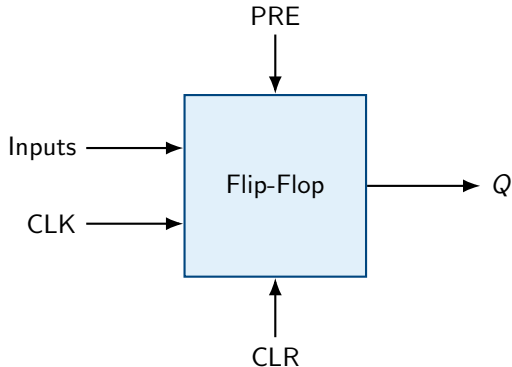
$$Q_{t+1} = J\overline{Q}_t + \overline{K}Q_t$$

Asynchronous Preset and Clear

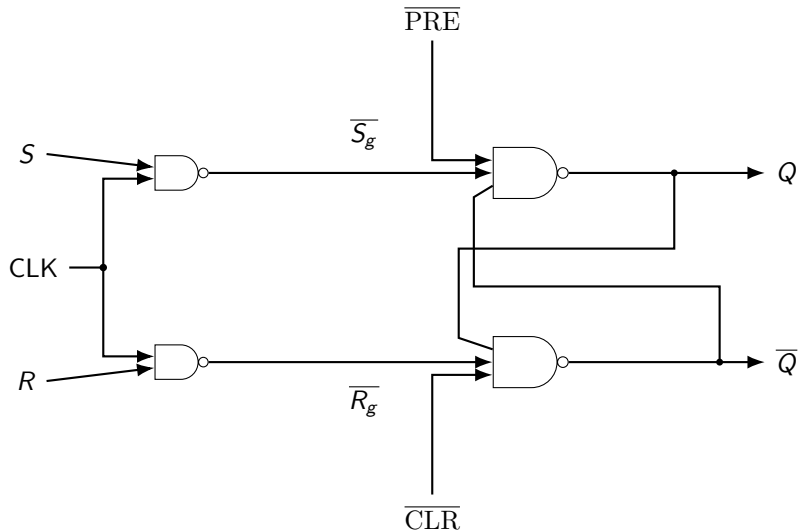
Normally, a flip-flop changes state according to its inputs and the **clock**.

But sometimes we need to initialize it immediately to a known state.

Preset (PRE) $\Rightarrow Q = 1$	Clear (CLR) $\Rightarrow Q = 0$
----------------------------------	---------------------------------



Clocked S-R Latch with Preset and Clear





M. Morris Mano and Michael D. Ciletti,
Digital Design, Pearson.

- **Chapter 4: Combinational Logic**
- **Chapter 5: Synchronous Sequential Logic**